

Prompts

1. We are building Enably: Go-to-Market Architect. It is a platform designed to assist users with setting up and managing a Go-to-Market architecture for their sales and marketing efforts. Focusing on Sales to start. We will build a Central Agent which is connected to the main dashboard and used as a support to help the user with any questions. An SDR assistant or the SDR agent itself, if the user is a small business with limited staff. It will take people through multiple processes to build and develop a Go-To-Market Engine, including setting up ideal customer profiles, using a conversational workflow assistant to ask you questions or allow you to upload materials and prompt, and get a dedicated ICP document with user personas, ICP break down, use cases industry research per ICP, unique selling propositions, as well as a sales playbook/ bible, with bdr/ae activities, SOPs, metrics, KPIs, tools, processes, how to use AI effectively, etc, a section /mage for messaging, email templates, phone scripts dm snippets, etc, as well as sales presentations (automations for those etc) . i need separate pages for this and a chat/assistant function that connects to our CRM to update, lead generation and prospecting, enrichment, etc and even create your own assistants with koweldgebases, powered by supabase backend for now. with open ai assistants api and agents sdk , responses api for open ai, with full threading and conversational history and setup. and a beautiful front end, back end , compoentns.
2. Create a go to market website application. The goal is to walk through the process of setting up a go to market engine or starting where you are. For example. There will be a section for refining or creating iCPs ideal customer profiles. A wizard to walk through or chat who you are what your product is what you want to sell why input company data etc. hand it will draft structured ICP document and use cases, and unique selling propositions and user personas and allow you to view and download them and allow you to save to a section with all your documents and files etc. organized well. And a BRR/Sales Bible and playbook, going over activity metrics and SOPs for outreach setting up email campaigns, messaging context, using AI instructions, everything to help the user, with a personal assistant that retains memory context for helping them with the chat and knowledge base upload so vector files or vectorization or maybe AWS s3 nucket integration and then messaging template creation, DM snippets etc. as well as custom research for each lead, company etc, enrichment integration. Connection to CRM or CRM connection integration, or create a new CRzm with leads etc. think of all the different factors it can help with a go to market strategy and create a dashboard architecture that I can build with v0 and update with GitHub etc to include the back end and allow custom integrations and MCPs and a section for users to create their own mini agents on the platform with a central sdr assistant. Not to replace the SDR but make their job easier if hand it off, so create courses and training etc. develop a plan for integrating this and developing it, core libraries and integrations I need, api keys, step by step instructions for backend and creating to front end and guide to make it work. Over his and step by step details and specifications PRD so I an build it tonight. Enably V1 application

openai

Create a go to market website application. The goal is to walk through the process of setting up a go to market engine or starting where you are. For example. There will be a section for refining or creating iCPs ideal customer profiles. A wizard to walk through or chat who you are what your product is what you want to sell why input company data etc. hand it will draft structured ICP document and use cases, and unique selling propositions and user personas and allow you to view and download them and allow you to save to a section with all your documents and files etc. organized well. And a BRR/Sales Bible and playbook, going over activity metrics and SOPs for outreach setting up email campaigns, messaging context, using AI instructions, everything to help the user, with a personal assistant that retains memory context for helping them with the chat and knowledge base upload so vector files or vectorization or maybe AWS s3 nucket integration and then messaging template creation, DM snippets etc. as well as custom research for each lead, company etc, enrichment integration. Connection to CRM or CRM connection integration, or create a new CRzm with leads etc. think of all the different factors it can help with a go to market strategy and create a dashboard architecture that I can build with v0 and update with GitHub etc to include the back end and allow custom integrations and MCPs and a section for users to create their own mini agents on the platform with a central sdr assistant. Not to replace the SDR but make their job easier if hand it off, so create courses and training etc. develop a plan for integrating this and developing it, core libraries and integrations I need, api keys, step by step instructions for backend and creating to front end and guide to make it work. Over his and step by step details and specifications PRD so I an build it tonight. Enably V1 application

response1

awesome—let's turn your idea into a shippable V1 you can start building tonight. below is a tight-but-complete PRD + architecture, step-by-step build plan, data models, UI wireframes, prompts, and integration stubs. you can trim or expand as you go, but this will get a functional MVP live quickly and keep it extensible.

product requirements (prd)

product vision

a self-serve go-to-market (gtm) operating system that helps founders and revenue teams define their icp, personas, usps, use-cases, and playbooks—then execute with templates, campaigns, research/enrichment, crm sync, and a memory-aware sdr assistant. not to replace sdrs—make them faster and more consistent.

primary users

- **founder / head of growth:** set strategy, review docs, configure integrations
- **sdr/ae:** use playbooks, generate messaging, run outreach tasks, log to crm
- **revops/admin:** manage data models, templates, permissions, integrations

top jobs-to-be-done (v1)

1. “i need a solid gtm foundation fast.” → icp/persona/usp wizard → downloadable artifacts
2. “give me a sales bible & outreach sop i can run tomorrow.” → playbook generator + activity metrics + cadences
3. “help me message each lead with context.” → research micro-agent + message generator + enrichment
4. “keep it organized & synced.” → document hub, crm sync, file store, memory-aware assistant

success metrics (v1)

- time-to-first-icp doc < 15 min
 - % users who export at least 3 assets in day 1 > 40%
 - first outreach sequence created < 30 min
 - crm sync configured within first session > 30%
-

scope: v1 features (build tonight)

1) onboarding + gtm wizard

- guided flow: company basics → product(s) → markets → value → proof → pricing → goals → channels
- outputs: structured **ICP**, **Personas**, **USPs**, **Use Cases**, **Messaging hierarchy**, **Positioning statement**
- actions: **Save**, **Edit**, **Export (PDF/Doc/JSON)**, **Promote to Playbook**

2) “sales bible” (brr) + playbook

- **activity model**: dials/email/LI DMs/meetings targets, SLAs, weekly review checklist
- **cadence templates**: email, LI DM, call openers
- **SOPs**: inbox mgmt, reply handling, objection handling, qualification (MEDDIC/BANT toggle)
- **live edit** + download

3) messaging workshop

- generator for: cold email v1–v3, LI DM, call opener, value bullets, problem-solution snippets
- supports **variables** from ICP/Persona/Company/Lead and **tone** presets

4) research & enrichment micro-agent

- inputs: lead or company url/domain → fetch summary (about, tech, news), key triggers, 3 talking points
- optional enrich via api key (clearbit, ppl/data vendor) — if absent, still do lightweight crawl + scrape

5) document hub

- store generated assets, uploads, templates; folders (by workspace/product)
- versioning (v1 simple “current + history”)

6) crm connection (v1: hubspot) + generic csv import/export

- oauth to hubspot; sync companies/contacts/notes; push activities from playbook
- csv import for leads; export sequences

7) assistant with memory

- chat with retrieval over workspace artifacts + uploads; remembers company/product context
 - note: memory scopes: **user**, **workspace**, **thread**
-

out of scope (v1) → v1.5/v2

- full salesforce integration, apollo outreach send, sequencing engine, dialer
 - course marketplace / certifications (provide 3 starter courses in v1 as static content)
 - multi-product P&L modeling
-

information architecture (ia)

top nav: Dashboard · Wizard · Playbook · Messaging · Research · Hub (Docs) · Integrations · Assistant

dashboard (cards):

- “finish your icp” progress • “create first cadence” CTA • “connect crm” status • recent docs • tasks for today

wizard:

- stepper with autosave; preview panel shows live doc outline and score (completeness)

playbook:

- left: sections (activity targets, sops, cadences)
- right: editor + variables sidebar (from icp/persona)

messaging:

- template library → generate → test send (copy to clipboard) → save as template

research:

- input domain/linkedin url → fetch → snapshot card → add to lead → generate custom message

hub (docs):

- folders, tags, search; view/export/version

assistant:

- chat with “context toggles” (icp / playbook / current lead); cite sources from hub

integrations:

- hubspot oauth, s3 bucket connect, embeddings provider, enrichment provider
-

system architecture

suggested stack (fastest path)

- **frontend:** Next.js 14 (App Router), TypeScript, Tailwind, shadcn/ui, Zustand (light), React Hook Form + Zod
- **auth & orgs:** Clerk (Orgs + SSO) *or* NextAuth + Lucia (Clerk is faster)
- **backend:** Next.js API routes (edge for chat; node for integrations) + tRPC (optional)
- **db:** Postgres (Neon/Supabase) + Prisma
- **vector store:** pgvector in same Postgres (simple + cheap)
- **cache/queue:** Upstash Redis (rate-limits, background jobs)
- **file storage:** AWS S3 (or Supabase storage)
- **observability:** OpenTelemetry + Logtail or Sentry
- **ai:** OpenAI (GPT-4.1/4o-mini for gen, text-embedding-3-large for embeddings)
- **crawler:** simple serverless fetch + Mercury Parser or Metascraper; add ScrapingBee key if needed

- **pdf export:** @react-pdf/renderer or server-side Puppeteer

alt: backend as **NestJS** + GraphQL if you prefer stricter boundaries; for tonight, Next.js monorepo is quicker.

high-level diagram (text)

client (next) → api routes

api routes → prisma/postgres, s3, hubspot oauth, enrichment api, openai

research worker (edge function/cron) → fetch page → summarize → store

assistant → r.agents: retrieve (pgvector) → prompt → stream back

data model (prisma snippets)

prisma

CopyEdit

```
model Workspace {
  id          String    @id @default(cuid())
  name        String
  slug        String    @unique
  createdAt   DateTime  @default(now())
  members     Member[]
  products    Product[]
  documents   Document[]
  companies   Company[]
  leads       Lead[]
  templates   Template[]
  agents      Agent[]
  integrations Integration?
}
```

```
model Member {
  id          String    @id @default(cuid())
  userId      String    // from Clerk
  role        Role
  workspaceId String
```

```
    Workspace    Workspace @relation(fields: [workspaceId], references:
[id])
}
```

```
enum Role { OWNER ADMIN SDR VIEWER }
```

```
model Document {
  id          String    @id @default(cuid())
  workspaceId String
  type        DocType
  title       String
  content      Json      // normalized JSON (see schemas below)
  version     Int        @default(1)
  tags        String[]
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
}
```

```
enum DocType { ICP PERSONA USP USECASE PLAYBOOK MESSAGE_TEMPLATE SOP
COURSE }
```

```
model Product {
  id          String    @id @default(cuid())
  workspaceId String
  name        String
  description  String?
  pricing     Json?
}
```

```
model Company {
  id          String    @id @default(cuid())
  workspaceId String
  name        String
  domain      String?   @unique
  size        Int?
  industry    String?
  tech        String[]
  notes       String?
```

```

    enrichedAt    DateTime?
}

model Lead {
  id              String    @id @default(cuid())
  workspaceId    String
  companyId       String?
  firstName      String
  lastName       String
  email          String?
  title          String?
  linkedinUrl    String?
  status         LeadStatus @default(NEW)
  fields         Json?
}

enum LeadStatus { NEW WORKING QUALIFIED DISQUALIFIED }

model Template {
  id              String    @id @default(cuid())
  workspaceId    String
  name           String
  channel        Channel
  body           String      // handlebars-style
  variables      String[]    // ["{{company_name}}", "{{pain_point}}", ...]
}

enum Channel { EMAIL LINKEDIN CALL }

model Agent {
  id              String    @id @default(cuid())
  workspaceId    String
  name           String
  purpose        String
  instructions    String
  tools          Json        // {webSearch:boolean, enrichment:boolean,...}
  memoryScope    String      // user/workspace/thread
}

```

```

model Integration {
  id          String    @id @default(cuid())
  workspaceId String    @unique
  hubspot     Json?
  s3          Json?
  enrichment  Json?
  embeddings  Json?
}

```

```

model Embedding {
  id          String    @id @default(cuid())
  workspaceId String
  docId       String?
  chunkId     String?
  content     String
  vector      Vector    // pgvector
  metadata    Json
}

```

normalized JSON schemas (stored in `Document.content`)

ICP

```

json
CopyEdit
{
  "name": "Mid-Market SaaS Security Buyers",
  "firmographics": {"employees": 200-1500, "industries":
["SaaS", "Fintech"], "regions": ["US", "UK"]},
  "technographics": ["Okta", "AWS", "Snowflake"],
  "pain_points": ["shadow it", "slow audits"],
  "value_hypotheses": ["reduce audit prep from weeks to hours"],
  "buying_committee": [{"role": "CISO", "influence": "driver"},
{"role": "Security Engineer", "influence": "blocker"}],
  "triggers": ["SOC2 renewal", "security incident", "new CISO"],
  "disqualifiers": ["<50 employees", "on-prem only"]
}

```

Persona

json

CopyEdit

```
{
  "role": "CISO",
  "goals": ["reduce risk", "pass audits"],
  "kpis": ["time-to-audit", "incident count"],
  "objections": ["data residency", "implementation time"],
  "messages": {"problem": "...", "value": "...", "proof": "..."},
  "preferred_channels": ["email", "linkedin"]
}
```

Playbook

json

CopyEdit

```
{
  "activity_targets":
{"daily":{"emails":40,"li_dms":20,"calls":30},"weekly":{"meetings":10}
},
  "sops": [{ "name": "inbox
management", "steps": [ "triage", "label", "respond" ] },
  "qualification": "MEDDICC",
  "cadences": [
    { "name": "Cold Email
7-touch", "steps": [ { "day": 0, "channel": "EMAIL", "template": "CE1" },
{ "day": 2, "channel": "EMAIL", "template": "CE2" } ] }
  ]
}
```

api surface (next.js route handlers)

swift

CopyEdit

/api/wizard/compile (POST) -> takes wizard inputs -> returns JSON docs
(ICP, Personas, USP, UseCases)

```
/api/docs (GET,POST)
/api/docs/:id (GET,PATCH,DELETE)
/api/templates (GET,POST)
/api/research/snapshot (POST) -> {domain|linkedinUrl} -> {summary,
tech, news, triggers}
/api/messaging/generate (POST) -> {templateId or raw, variables, tone}
/api/integrations/hubspot/oauth (GET) -> redirect
/api/integrations/hubspot/sync (POST) ->
{entity:"contacts"|"companies"|"notes"}
/api/assistant/chat (POST) -> RAG over embeddings + memory scope
```

ai: prompt blueprints

wizard → documents

makefile

CopyEdit

SYSTEM: You are a GTM architect. Produce concise, structured JSON for {doc_type}.

Include only fields present in the provided JSON schema.

USER CONTEXT:

Company: {company_overview}

Product(s): {products}

Goals: {goals}

Markets: {markets}

Proof: {social_proof}

Pricing: {pricing}

Constraints: {constraints}

OUTPUT: Valid JSON per schema.

messaging generator

pgsql

CopyEdit

SYSTEM: You are a world-class SDR copywriter.
Write {channel} outreach that is personalized using VARIABLES and consistent with the ICP and Persona.

VARIABLES: {kv_pairs}

TONE: {tone} // choices: direct, friendly, challenger, technical

CONSTRAINTS: 90-120 words max for email; 200 characters for DM;
No fluff. Include 1 value prop and 1 CTA.

OUTPUT: Raw text with placeholders resolved.

research micro-agent

arduino

CopyEdit

SYSTEM: You analyze public sources and extract signals useful for cold outreach.

INPUTS: company_url={url} domain={domain} about_html={text}
news_snippets={list}

TASKS:

- 1) 1-sentence company summary.
- 2) 3 recent signals or triggers (with source).
- 3) Product/tech hints (stack).
- 4) 3 custom talking points mapped to our ICP value.

OUTPUT: JSON {summary, signals:[{text,source}], tech:[...],
talking_points:[...]}

assistant (RAG)

- chunk normalized docs (512-1k tokens), embed with text-embedding-3-large, store in pgvector
- retrieval: `SELECT ... ORDER BY vector <-> query LIMIT 8;`

- prompt includes **citations** by document title + section
-

build plan (tonight)

0) repo + packages

bash

CopyEdit

```
npx create-next-app gtm-os --typescript --eslint --tailwind --app
cd gtm-os
pnpm add @clerk/nextjs @tanstack/react-query zod react-hook-form \
  prisma @prisma/client pg pgvector drizzle-orm # (or just Prisma) \
  openai langchain jsdom metascraper puppeteer \
  @vercel/og @react-pdf/renderer \
  aws-sdk @aws-sdk/client-s3 \
  @upstash/redis @upstash/qstash \
  lucide-react class-variance-authority clsx \
  next-safe-action @sentry/nextjs
pnpm dlx prisma init
```

1) database & vector

- use Neon or Supabase Postgres
- enable pgvector:

sql

CopyEdit

```
CREATE EXTENSION IF NOT EXISTS vector;
```

- add prisma schema (above), run:

bash

CopyEdit

```
pnpm dlx prisma migrate dev --name init
```

2) auth & orgs

- Clerk: create app → get `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`
- wrap `<ClerkProvider>` in `app/layout.tsx`; protect routes with `auth()` or `<SignedIn>`

3) storage

- create S3 bucket `gtm-os-{env}`; put creds in `.env`

4) environment (.env.local)

ini

CopyEdit

```
DATABASE_URL=postgres://...
```

```
OPENAI_API_KEY=...
```

```
EMBEDDING_MODEL=text-embedding-3-large
```

```
S3_ACCESS_KEY_ID=...
```

```
S3_SECRET_ACCESS_KEY=...
```

```
S3_BUCKET=gtm-os-dev
```

```
S3_REGION=us-east-1
```

```
CLERK_SECRET_KEY=...
```

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=...
```

```
HUBSPOT_CLIENT_ID=...
```

```
HUBSPOT_CLIENT_SECRET=...
```

```
HUBSPOT_REDIRECT_URI=http://localhost:3000/api/integrations/hubspot/oauth/callback
```

```
UPSTASH_REDIS_REST_URL=...
```

```
UPSTASH_REDIS_REST_TOKEN=...
```

5) api routes (skeletons)

/app/api/wizard/compile/route.ts

ts

CopyEdit

```
import { NextRequest, NextResponse } from 'next/server'
import OpenAI from 'openai'
import { z } from 'zod'
const oai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! })

export async function POST(req: NextRequest) {
  const body = await req.json()
  // body: {company, products, goals, markets, proof, pricing}
  const docs = await generateDocs(body) // call oai with prompt
  blueprints
  // persist as Document rows
  return NextResponse.json(docs)
}
```

/app/api/messaging/generate/route.ts

ts

CopyEdit

```
export async function POST(req: Request) {
  const { templateBody, variables, tone="direct", channel="EMAIL" } =
  await req.json()
  const text = await generateMessage({ templateBody, variables, tone,
  channel })
  return Response.json({ text })
}
```

/app/api/research/snapshot/route.ts

ts

CopyEdit

```
import metascraper from 'metascraper'
export async function POST(req: Request) {
  const { domain, url } = await req.json()
  const html = await fetch(url ||
  `https://${domain}`).then(r=>r.text())
```

```
const parsed = /* extract title/description/tech via heuristics */
const news = [] // stub; add scrapingbee later
const json = await summarizeResearch({ html, parsed, news })
return Response.json(json)
}
```

/app/api/assistant/chat/route.ts

ts

CopyEdit

```
// do retrieval from pgvector, feed to oai, stream response
```

/app/api/integrations/hubspot/...

- oauth handler; store tokens in `Integration.hubspot`
- simple sync endpoint to upsert companies/contacts

6) ui scaffolding

- shadcn/ui: install Button, Card, Tabs, Accordion, Dialog, Sheet, Badge, Table, Progress, Stepper
- pages:
 - `/dashboard` – cards + progress
 - `/wizard` – stepper form; right panel “Live Preview”
 - `/playbook` – editor with sections (activity, SOPs, cadences)
 - `/messaging` – template library + generator pane
 - `/research` – input domain/url → snapshot card → “Add to lead”
 - `/hub` – file tree + search
 - `/integrations` – oauth buttons

- `/assistant` – chat with context toggles

7) embeddings + rag

- chunk each saved Document; embed and store in `Embedding`
- retrieval function for assistant and for “smart insert variables” in messages

8) exports

- PDF renderer for ICP/Persona/Playbook (header, metadata, sections)
- JSON export (raw schemas)

9) roles & multi-tenancy

- Clerk org id = `workspaceId`; scope all queries by `workspaceId`
-

minimal ui wireframe notes (fast)

- **wizard**: left column stepper; center form; right preview (JSON → neat outline)
 - **playbook**: tabs: Activity | SOPs | Cadences; each step editable; “Insert from ICP” button
 - **messaging**: left list of templates; right editor; variables drawer with autocomplete
 - **research**: search bar; results grid with “signals” chips; “Generate email from this” button
 - **assistant**: chat + “attach docs” and “use current ICP” toggles
-

integrations (v1)

hubspot oauth

- use `@hubspot/api-client`
- scopes: `crm.objects.contacts.read/write`,
`crm.objects.companies.read/write`, `crm.schemas.custom.read`
- flow: `/integrations/hubspot/connect` → redirect → save tokens
- sync functions:
 - **push note** with generated message or meeting
 - **upsert contact/company** by email/domain

enrichment (optional in v1)

- support Clearbit (domain lookup) if key present; otherwise no-key fallback parser.

storage (s3)

- folder per workspace → `/workspace/{id}/docs/{docId}.json` and
`/exports/{docId}.pdf`

sample templates

email – problem/trigger/value

CSS

CopyEdit

Subject: `{Trigger}` at `{{company_name}}`

Hi {{first_name}}, noticed {{trigger_observation}} at {{company_name}}. Teams like {{peer_1}} cut {{pain_metric}} by {{proof_stat}} using {{product}}.

Worth a 12-min call to see if we can {{value_outcome}}?
– {{sender}}

LI DM – short

css

CopyEdit

Hey {{first_name}} – saw you’re using {{tech}}. We help {{persona_role}} reduce {{pain}}. Open to a quick compare notes?

call opener

bash

CopyEdit

“{{first_name}} it’s {{sender}} – 15 seconds, then you tell me if we continue. We help {{persona_role}} at {{industry}} companies cut {{pain}}. Should I share why {{peer}} cared?”

“mini-agents” (user-created)

Schema in [Agent](#) model. UI to create:

- name, purpose, instructions
- tool toggles: “Use ICP”, “Use Research”, “Web fetch”, “CRM lookup”
- memory scope: thread/workspace
- execution: compose a system prompt from toggles + instructions, then call OpenAI.
Starter agents:
 1. **ICP Refiner** – critiques and tightens ICP JSON

2. **Persona Copy Chef** – writes variant messaging per persona
 3. **Research Scout** – finds 3 triggers and drafts opener
 4. **Objection Handler** – suggests responses + proof points
-

security & compliance (v1 basics)

- row-level authorization by `workspaceId`
 - encrypt integration secrets at rest
 - signed s3 upload urls
 - rate limit AI endpoints with Upstash (e.g., 60/min/user)
-

developer experience

- monorepo only (app + prisma)
 - add GitHub Actions: lint, typecheck, prisma migrate, preview deploy (Vercel)
 - seed script for demo workspace with dummy ICP/Persona
 - feature flags via simple `config` table or environment
-

timelines & milestones

tonight (v1 “enable”):

- auth/orgs, db schema, wizard (form + compile + save), hub (list/view/export), messaging generator, assistant (RAG minimal), research (basic), hubspot connect (store token; stub sync), s3 uploads, 3 starter cadences, 3 starter courses.

next 72 hours:

- enrichment plug, activity dashboard, pdf polish, robust error states, user-created agents ui, crm sync job.
-

copy/paste starters

generate docs (oai call)

ts

CopyEdit

```
async function oaiJson(system: string, user: string) {
  const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! })
  const res = await openai.chat.completions.create({
    model: "gpt-4.1-mini",
    response_format: { type: "json_object" },
    messages: [{ role: "system", content: system }, { role: "user",
content: user }]
  })
  return JSON.parse(res.choices[0].message.content!)
}
```

embed + store

ts

CopyEdit

```
async function embedAndStore(workspaceId: string, docId: string, text:
string) {
  const openai = new OpenAI()
  const { data } = await openai.embeddings.create({
    model: "text-embedding-3-large",
    input: text
  })
}
```

```
  })  
  // insert into Embedding with prisma  
}
```

handlebars-style variable fill

ts

CopyEdit

```
function fill(template: string, vars: Record<string,string>) {  
  return template.replace(/\{\{(.*)\}\}/g, (_, k) => vars[k.trim()]  
  ?? "")  
}
```

quick start checklist (order)

1. create next app + install deps
 2. set up prisma models + migrate
 3. add clerk + orgs; secure routes
 4. build **wizard** form (zod schemas) + `/api/wizard/compile`
 5. implement **document hub** (list/view/export)
 6. add **messaging** page with generator + variables
 7. wire **embeddings** on document save; implement **assistant** chat with RAG
 8. build **research** endpoint + simple ui
 9. add **hubspot** oauth save (no full sync tonight)
 10. deploy to vercel; connect neon; set envs; test export
-

training content (seed)

- “how to run your first 2 weeks of outbound” (course)
 - “how to write 5-message cadence per persona” (course)
 - “how to run weekly pipeline review” (checklist)
-

core libraries (concise)

- **ui:** next, tailwind, shadcn/ui
 - **forms:** react-hook-form, zod
 - **data:** prisma, postgres, pgvector
 - **ai:** openai sdk
 - **auth:** clerk
 - **files:** aws s3
 - **crm:** @hubspot/api-client
 - **scrape:** metascraper/jsdom
 - **pdf:** @react-pdf/renderer
 - **ops:** sentry, upstash redis
-

want me to tailor the wizard questions and default templates to your specific product and market? if you share a quick blurb (what you sell + to whom), i'll prefill the ICP/persona schemas and first cadence so you've got a demo-ready flow.

page directory

awesome—let's map the pages first, then give you a clean Next.js (App Router) directory structure you can paste in and scaffold.

pages (routes) you need

1. Dashboard

- `/dashboard` — overview cards (progress, tasks, recent docs, CRM status)

2. Wizard (GTM foundation)

- `/wizard` — stepper shell
- `/wizard/icp`
- `/wizard/personas`
- `/wizard/usps`
- `/wizard/use-cases`
- `/wizard/messaging-hierarchy`
- `/wizard/positioning`
- `/wizard/review` (compile → save → export)

3. Playbook (“Sales Bible”)

- `/playbook` — tabs: Activity Targets | SOPs | Cadences | Qualification
- `/playbook/editor` — full-screen editor (optional route)

4. Messaging

- `/messaging` — template library + generator panel
- `/messaging/templates/[id]` — edit/view template

5. Research & Enrichment

- `/research` — input domain/LI URL → snapshot cards
- `/research/leads/[id]` — enriched lead view + draft messages

6. Hub (Docs & Files)

- `/hub` — folders, tags, search
- `/hub/docs/[id]` — view/export/version
- `/hub/uploads` — file manager

7. Assistant (Memory/RAG)

- `/assistant` — chat with context toggles (ICP/Playbook/Current Lead)

8. Integrations

- `/integrations` — list + status
- `/integrations/hubspot` — connect & sync controls
- `/integrations/storage` — S3 connection
- `/integrations/enrichment` — Clearbit/other keys

9. Admin / Settings

- `/settings` — workspace/profile, roles, feature flags

10. Auth (Clerk auto-routes)

- `/sign-in`, `/sign-up`, `/orgs` (handled by Clerk components)

11. API routes

- `/api/wizard/compile`
- `/api/docs` & `/api/docs/[id]`

- `/api/templates & /api/templates/[id]`
 - `/api/research/snapshot`
 - `/api/messaging/generate`
 - `/api/assistant/chat` (RAG)
 - `/api/integrations/hubspot/{connect,callback,sync}`
 - `/api/storage/sign` (S3 signed URLs)
-

directory structure (Next.js App Router + feature folders)

Paste this in your repo planning docs — it's pragmatic and scales:

bash

CopyEdit

```
/app
  /api
    /assistant
      /chat/route.ts
    /docs
      route.ts           # GET, POST
      /[id]/route.ts     # GET, PATCH, DELETE
    /integrations
      /hubspot
        /connect/route.ts
        /callback/route.ts
        /sync/route.ts
    /messaging
      /generate/route.ts
    /research
      /snapshot/route.ts
    /storage
```

```
    /sign/route.ts
  /templates
    route.ts
    /[id]/route.ts
  /wizard
    /compile/route.ts
/(public)
  /dashboard
    page.tsx
  /wizard
    layout.tsx
    page.tsx          # stepper shell + navigation
    /icp/page.tsx
    /personas/page.tsx
    /usps/page.tsx
    /use-cases/page.tsx
    /messaging-hierarchy/page.tsx
    /positioning/page.tsx
    /review/page.tsx
  /playbook
    page.tsx
    /editor/page.tsx
  /messaging
    page.tsx
    /templates
      page.tsx        # list
      /[id]/page.tsx  # editor
  /research
    page.tsx
    /leads
      /[id]/page.tsx
  /hub
    page.tsx
    /docs
      /[id]/page.tsx
    /uploads/page.tsx
  /assistant
    page.tsx
```

```
/integrations
  page.tsx
  /hubspot/page.tsx
  /storage/page.tsx
  /enrichment/page.tsx
/settings
  page.tsx
layout.tsx
globals.css

/components
  ui/                                # shadcn components (Button, Card, Tabs,
etc.)                                etc.)
  common/
    PageHeader.tsx
    EmptyState.tsx
    DataTable.tsx
    JsonPreview.tsx
    Stepper.tsx
    TagInput.tsx
  dashboard/
    ProgressCards.tsx
    QuickActions.tsx
  wizard/
    WizardForm.tsx
    IcPForm.tsx
    PersonasForm.tsx
    UspsForm.tsx
    UseCasesForm.tsx
    MessagingHierarchyForm.tsx
    PositioningForm.tsx
    ReviewPanel.tsx
  playbook/
    ActivityEditor.tsx
    SopEditor.tsx
    CadenceEditor.tsx
    QualificationPicker.tsx
  messaging/
```

```
    TemplateList.tsx
    TemplateEditor.tsx
    GeneratorPanel.tsx
    VariablesDrawer.tsx
research/
  ResearchBar.tsx
  SnapshotCard.tsx
  LeadActions.tsx
hub/
  FileTree.tsx
  DocViewer.tsx
  ExportMenu.tsx
assistant/
  ChatWindow.tsx
  ContextToggles.tsx
  MessageBubble.tsx
integrations/
  HubspotConnect.tsx
  StorageConnect.tsx
  EnrichmentKeys.tsx

/lib
  ai/
    prompts.ts
    openai.ts
    embeddings.ts
    rag.ts
  db/
    prisma.ts
    queries.ts
  storage/
    s3.ts
  crm/
    hubspot.ts
  enrichment/
    clearbit.ts
    scrape.ts
  pdf/
```

```
    exporter.tsx
auth/
  clerk.ts
utils/
  logger.ts
  format.ts
  server-safe-action.ts

/hooks
  useWizard.ts
  useToast.ts
  usePagination.ts
  useDebounce.ts
  useUpload.ts

/types
  docs.ts                # ICP/Persona/Playbook JSON schemas (TS
types)
  api.ts
  common.ts

/prisma
  schema.prisma
  seed.ts

/styles
  globals.css
  prose.css
  shadcn.css

/public
  logo.svg
  icons/*
  demo/*

/config
  featureFlags.ts
  app.ts                # constants (limits, defaults)
```

```
/test
  e2e/*
  unit/*
```

```
.env.example
```

quick stubs (copy/paste)

/app/(public)/dashboard/page.tsx

```
tsx
CopyEdit
export default function Dashboard() {
  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-semibold">Dashboard</h1>
      { /* <ProgressCards/> <QuickActions/> */ }
    </div>
  )
}
```

/app/(public)/wizard/page.tsx (shell)

```
tsx
CopyEdit
import Stepper from "@/components/common/Stepper"
export default function Wizard() {
  const steps = ["ICP", "Personas", "USPs", "Use Cases", "Messaging", "Positioning", "Review"]
  return (
    <div className="grid grid-cols-12 gap-6">
      <aside className="col-span-3"><Stepper steps={steps}
current={0}/></aside>
      <main className="col-span-9">Select a step on the left</main>
    </div>
  )
}
```

```
}
```

/app/api/wizard/compile/route.ts

```
ts
```

```
CopyEdit
```

```
import { NextRequest, NextResponse } from "next/server"
import { compileWizard } from "@lib/ai/prompts"

export async function POST(req: NextRequest) {
  const body = await req.json()
  const docs = await compileWizard(body) // returns {icp, personas,
  usps, useCases, messaging, positioning}
  // TODO: persist Documents via prisma
  return NextResponse.json(docs)
}
```

/lib/ai/prompts.ts (shape)

```
ts
```

```
CopyEdit
```

```
import OpenAI from "openai"
const oai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! })

export async function compileWizard(inputs: any) {
  // call model per doc type using schemas; return combined JSON
  return { icp:{}, personas:[], usps:[], useCases:[], messaging:{},
  positioning:{} }
}
```

notes on structure & conventions

- **Feature-first:** routes, components, and libs are grouped by feature; keeps contexts small and testable.
- **Colocation:** small feature-only hooks/components can live under their feature folder.

- **App Router:** use nested `layout.tsx` where a feature needs persistent UI (e.g., Wizard stepper).
- **Server actions/API:** start with API routes as above; migrate hot paths to server actions later if you prefer.
- **Types/Schemas:** define Zod schemas in `/types/docs.ts` and infer TS types from them for solid forms.

want me to generate empty files and minimal contents as a script so you can `pnpm scaffold` and get all folders created? I can output a ready-to-run bash script next.

cline

1. We are building Enably: Go-to-Market Architect. It is a platform designed to assist users with setting up and managing a Go-to-Market architecture for their sales and marketing efforts. Focusing on Sales to start. We will build a Central Agent which is connected to the main dashboard and used as a support to help the user with any questions. An SDR assistant or the SDR agent itself, if the user is a small business with limited staff. It will take people through multiple processes to build and develop a Go-To-Market Engine, including setting up ideal customer profiles, using a conversational workflow assistant to ask you questions or allow you to upload materials and prompt, and get a dedicated ICP document with user personas, ICP break down, use cases industry research per ICP, unique selling propositions, as well as a sales playbook/ bible, with bdr/ae activities, SOPs, metrics, KPIs, tools, processes, how to use AI effectively, etc, a section /mage for messaging, email templates, phone scripts dm snippets, etc, as well as sales presentations (automations for those etc) . i need separate pages for this and a chat/assistant function that connects to our CRM to update, lead generation and prospecting, enrichment, etc and even create your own assistants with koweldgebases, powered by supabase backend for now. with open ai assistants api and agents sdk , responses api for open ai, with full threading and conversational history and setup. and a beautiful front end, back end , compoentns.

PRD

__Project: Enably - The Go-to-Market OS (V1)__

__1. Product Requirements Document (PRD)__

__1.1. Vision__ A self-serve, AI-powered Go-to-Market (GTM) operating system. Enably empowers founders, sales, and marketing teams to rapidly build and execute a world-class GTM strategy. It starts by defining a strategic foundation (ICP, Personas, USPs) and flows into tactical execution with playbooks, messaging, and an intelligent SDR assistant. The goal is not to replace humans, but to make them faster, smarter, and more consistent.

__1.2. Target Users & Key Jobs-to-be-Done (JTBD)__

- __Founder / Head of Growth:__

- __JTBD:__ "I need a solid GTM foundation, fast."

- __Solution:__ A guided wizard to generate foundational documents (ICP, Personas, USPs).

- __SDR / AE (Sales Development Rep / Account Executive):__

- __JTBD:__ "Give me a 'sales bible' and clear steps for outreach."

- __Solution:__ A dynamic Sales Playbook with activity targets, SOPs, and messaging templates.

- __JTBD:__ "Help me write relevant, personalized messages to every lead."

- __Solution:__ A research micro-agent and a context-aware message generator.

- __RevOps / Admin:__

- __JTBD:__ "Keep our GTM assets organized, synced, and secure."

- __Solution:__ A centralized document hub, CRM integration, and user/workspace management.

__1.3. V1 Scope & Core Features__

1. __Onboarding & GTM Wizard:__ A step-by-step conversational flow to capture company, product, and market details.

- __Outputs:__ Structured, downloadable documents for Ideal Customer Profile (ICP), Personas, Unique Selling Propositions (USPs), and Use Cases.

2. __The Sales Bible & Playbook:__ A central repository for sales strategy and operations.

- __Includes:__ Activity targets (dials, emails), Standard Operating Procedures (SOPs), and outreach cadence templates.

3. __Messaging Workshop:__ An AI-powered generator for crafting outreach copy (emails, LinkedIn DMs, call scripts) using variables from the GTM foundation.

4. __Research & Enrichment Micro-Agent:__ A tool to fetch and summarize key information about a lead or company from a URL, providing talking points for outreach.

5. __Document Hub:__ A central, versioned repository for all generated assets and user uploads, organized by workspace.
6. __CRM Integration (V1: HubSpot):__ OAuth connection to sync contacts, companies, and notes, enabling a seamless workflow between Enably and the user's CRM.
7. __Memory-Aware Assistant:__ An AI chat assistant with retrieval-augmented generation (RAG) capabilities, allowing it to answer questions based on the user's documents and workspace context.

__1.4. Success Metrics (V1)__

- __Activation:__ Time-to-first-ICP document < 15 minutes.
- __Engagement:__ > 40% of users export at least 3 assets within their first day.
- __Adoption:__ > 30% of users connect their CRM during their first session.

__2. Project Overview & Architecture__

__2.1. System Architecture & Tech Stack__

- __Frontend:__ __Next.js 14__ (App Router), TypeScript, Tailwind CSS.
- __UI Components:__ __shadcn/ui__ for a modern, accessible, and consistent design system.
- __State Management:__ __Zustand__ for lightweight global state.
- __Forms:__ __React Hook Form__ with __Zod__ for validation.
- __Backend:__ __Supabase__
 - __Database:__ Postgres with __pgvector__ for embeddings.
 - __Auth:__ Supabase Auth for user and organization management.
 - __Storage:__ Supabase Storage for file uploads (documents, assets).
 - __Serverless Functions:__ Supabase Edge Functions for backend logic (e.g., AI-powered generation, CRM sync).
- __ORM:__ __Prisma__ for type-safe database access.
- __AI:__ __OpenAI API__ (`gpt-4o-mini` for generation, `text-embedding-3-large` for embeddings).
- __Web Scraping:__ __Metascraper__ and `jsdom` for the research agent.
- __PDF Generation:__ __@react-pdf/renderer__ for exporting documents.

__2.2. Information Architecture (IA)__

- __Top Navigation:__ Dashboard | Wizard | Playbook | Messaging | Research | Hub | Integrations | Assistant.
- __Dashboard:__ An overview of key activities, progress, and quick-access links.
- __Data Models:__ A robust Prisma schema defining `Workspace`, `Member`, `Document`, `Product`, `Lead`, `Template`, and `Integration` relationships, ensuring data integrity and multi-tenancy.

__2.3. High-Level Build Plan__

1. __Phase 1: Foundation (The "Enable" Sprint)__

- __Setup:__ Initialize Next.js project, install all dependencies.
- __Database:__ Set up Supabase project, enable `pgvector`, and run initial Prisma migrations based on the defined schema.
- __Auth & Orgs:__ Configure Supabase Auth for user sign-up, login, and workspace management.
- __Core UI:__ Scaffold the main application layout, navigation, and basic pages using `shadcn/ui`.

2. __Phase 2: The Wizard & Document Hub__

- Build the multi-step GTM Wizard form.
- Create the backend API endpoint to take wizard inputs, call the OpenAI API, and generate structured JSON documents.
- Implement the Document Hub to list, view, and export these generated documents (PDF & JSON).

3. __Phase 3: Playbook & Messaging__

- Develop the UI for the Sales Playbook and Messaging Workshop.
- Create the AI-powered message generator.

4. __Phase 4: Assistant & Integrations__

- Implement the embedding process on document save.
- Build the RAG-powered chat assistant.
- Set up HubSpot OAuth and initial data sync functionality.

This refined plan provides a clear, actionable path to building the V1 of Enably. It leverages the power of Supabase for a streamlined backend experience while following the excellent, detailed architecture you provided.

Build plan'

Build plan'

__Phase 1: Backend & Core Agent Infrastructure__

1. __Dependencies & Setup:__

- __Install:__ I'll install all necessary libraries: `@supabase/auth-helpers-nextjs`, `@supabase/supabase-js`, `ai` (for the Vercel AI SDK), `openai`, and `@google/generative-ai`.
- __Environment:__ I will update the `.env` file with placeholders for Supabase, OpenAI, Gemini, and the external data APIs (like Apify, Hunter.io, and SerpAPI) that you will need to provide.

2. __Supabase Database Schema:__

- I will create the database schema in Supabase as defined in your documents. This includes three core tables:
 - `jobs`: To track user prompts, agent type, and status.
 - `leads`: To store the structured contact and company data found by the Prospector Agent.
 - `lead\_insights`: To store the rich, personalized data generated by the Research Assistant.

3. __Agent API Endpoints:__

- __Prospector Agent (`/api/prospector`):__ This endpoint will receive a natural language prompt. It will use AI function-calling (OpenAI or Gemini) to parse the prompt and then orchestrate calls to external APIs (Apify, etc.) to find and enrich leads, finally saving them to the `leads` table.
- __Research Assistant (`/api/research`):__ This endpoint will take a `jobId`, retrieve the associated leads, and use AI to research them, generating and saving strategic insights, quotes, and messaging angles to the `lead\_insights` table.

__Phase 2: Frontend UI & Agent Interaction__

- #### 1. __Authentication:__
- I will set up Supabase authentication (email/password or magic link) and create the necessary `login` and `signup` pages.

2. __Main Query Interface (`/`):__

- I will build the main page using a chat-based interface, as shown in your `ChatComponent` example. This will be the primary way users interact with both the Prospector and Research agents.
- I'll integrate `shadcn/ui` toast notifications to give users real-time feedback when a job is submitted, running, or complete.

3. __Results & Logs Pages:__

- __Results Page (`/results/[jobId]`):__ A dynamic page to view the outcome of a job. It will show a loading state while running and display the final lead list in a detailed table, including all enriched insights. A "Download CSV" button will be included.

- __Logs Page (`/logs`):__ A dashboard showing a history of all submitted jobs, their status, and links to the results.

__Phase 3: CRM & Advanced Features__

1. __Internal CRM (`/crm`):__

- I will build upon the `leads` and `lead\_insights` tables to create an internal CRM.
- This will feature both a __Table View__ and a __Card View__ for visualizing contacts.
- I'll create a __Contact Profile Page (`/crm/[id]`)__ that serves as the "extended profile," displaying all known information and insights for a single lead.

2. __GTM Document Storage:__ I will set up Supabase Storage for uploading and managing ICP and Playbook documents, as we discussed initially.

3. __External Integrations:__ As a final step, I will add the "Send to CRM" functionality to push leads to external platforms like Hubspot or Pipedrive.

This plan directly translates the vision from your documents into a concrete development roadmap. We will start by building the core agent and data infrastructure before moving to the user-facing components.

Does this new, detailed plan accurately capture your vision? If so, please __toggle to__

__Act Mode (⌘⇧A)__, and I will begin with Phase 1.

Thank you for providing such detailed documentation. This clarifies the vision for the project immensely. You're not just building a GTM asset manager, but a powerful, dual-agent lead generation and intelligence platform.

I have synthesized all the information from the documents and our previous conversations into a new, comprehensive plan that focuses on building this "Outbound Agent Ops" system.

Here is the revised, multi-phase plan:

__Phase 1: Backend & Core Agent Infrastructure__

1. __Dependencies & Setup:__

- __Install:__ I'll install all necessary libraries: `@supabase/auth-helpers-nextjs`, `@supabase/supabase-js`, `ai` (for the Vercel AI SDK), `openai`, and `@google/generative-ai`.
- __Environment:__ I will update the `.env` file with placeholders for Supabase, OpenAI, Gemini, and the external data APIs (like Apify, Hunter.io, and SerpAPI) that you will need to provide.

2. __Supabase Database Schema:__

- I will create the database schema in Supabase as defined in your documents. This includes three core tables:

- `jobs`: To track user prompts, agent type, and status.
- `leads`: To store the structured contact and company data found by the Prospector Agent.
- `lead\_insights`: To store the rich, personalized data generated by the Research Assistant.

3. __Agent API Endpoints:__

- __Prospector Agent (`/api/prospector`):__ This endpoint will receive a natural language prompt. It will use AI function-calling (OpenAI or Gemini) to parse the prompt and then orchestrate calls to external APIs (Apify, etc.) to find and enrich leads, finally saving them to the `leads` table.

- __Research Assistant (`/api/research`):__ This endpoint will take a `jobId`, retrieve the associated leads, and use AI to research them, generating and saving strategic insights, quotes, and messaging angles to the `lead\_insights` table.

__Phase 2: Frontend UI & Agent Interaction__

1. __Authentication:__ I will set up Supabase authentication (email/password or magic link) and create the necessary `login` and `signup` pages.

2. __Main Query Interface (`/`):__

- I will build the main page using a chat-based interface, as shown in your `ChatComponent` example. This will be the primary way users interact with both the Prospector and Research agents.

- I'll integrate `shadcn/ui` toast notifications to give users real-time feedback when a job is submitted, running, or complete.

3. __Results & Logs Pages:__

- __Results Page (`/results/[jobId]`):__ A dynamic page to view the outcome of a job. It will show a loading state while running and display the final lead list in a detailed table, including all enriched insights. A "Download CSV" button will be included.

- __Logs Page (`/logs`):__ A dashboard showing a history of all submitted jobs, their status, and links to the results.

__Phase 3: CRM & Advanced Features__

1. __Internal CRM (`/crm`):__

- I will build upon the `leads` and `lead\_insights` tables to create an internal CRM.
- This will feature both a __Table View__ and a __Card View__ for visualizing contacts.
- I'll create a __Contact Profile Page (`/crm/[id]`)__ that serves as the "extended profile," displaying all known information and insights for a single lead.

2. __GTM Document Storage:__ I will set up Supabase Storage for uploading and managing ICP and Playbook documents, as we discussed initially.

3. __External Integrations:__ As a final step, I will add the "Send to CRM" functionality to push leads to external platforms like Hubspot or Pipedrive.

This plan directly translates the vision from your documents into a concrete development roadmap. We will start by building the core agent and data infrastructure before moving to the user-facing components.

Tab 9

__Prompts for v0.dev to Build the Enably Homepage__

__Target Style:__ Modern, clean, dark mode, with a single prominent call-to-action. Use `shadcn/ui` components and `Tailwind CSS`.

__Prompt 1: Full Homepage Layout__

> "Create a full-page landing page for a SaaS product called 'Enably'. The design should be clean, modern, and inspired by suno.com. It needs a dark theme.

>

> The page should have three main sections:

>

> 1. __Hero Section:__ A full-height section with a large, bold headline: 'Build Your Go-to-Market Engine in Minutes.' Below the headline, add a sub-headline: 'Define your strategy, generate sales playbooks, and activate your team with our AI-powered GTM operating system.' The main call-to-action should be a large text input field with a button next to it that says 'Generate My Free Playbook'.

>

> 2. __Features Section:__ A section with the title 'Your Complete GTM Toolkit'. Use a 3-column grid of cards (`shadcn/ui Card`). Each card should have an icon, a title, and a short description.

>

> - Card 1: Icon: `Target`, Title: 'ICP & Persona Wizard', Description: 'Go from zero to a complete Ideal Customer Profile with our guided, conversational builder.'

> - Card 2: Icon: `BookOpen`, Title: 'Dynamic Sales Playbooks', Description: 'Generate comprehensive sales bibles with SOPs, scripts, and activity targets.'

> - Card 3: Icon: `Bot`, Title: 'AI SDR Assistant', Description: 'Augment your team with an assistant that can research leads, draft emails, and answer questions.'

>

> 3. __Social Proof Section:__ A simple, clean section displaying a row of 5-6 grayscale logos of well-known tech companies like 'HubSpot', 'Salesforce', 'Gong', 'Zendesk', and 'Intercom'.

>

> The top navigation bar should be simple, with the 'Enably' logo on the left and two buttons on the right: a secondary 'Login' button and a primary 'Sign Up' button."

__Prompt 2: Isolated Hero Section (If you want to build piece-by-piece)__

> "Create a full-screen hero section for a SaaS landing page using `shadcn/ui` and Tailwind CSS. It should have a dark background. In the center, include the H1 title 'Build Your Go-to-Market Engine in Minutes' and a p tag below it with the text 'Define your strategy, generate sales playbooks, and activate your team with our AI-powered GTM operating system.'

>

> Below the text, create a form element containing a large `shadcn/ui` Input component with the placeholder 'Describe your core product...' and a `shadcn/ui` Button next to it with the text 'Generate My Free Playbook'."

__Prompt 3: Isolated Features Section__

> "Create a features section with a dark theme. The section should have a centered H2 title 'Your Complete GTM Toolkit'. Below the title, create a responsive 3-column grid. Each item in the grid should be a `shadcn/ui` Card component. Each card needs a `lucide-react` icon at the top, a CardTitle, and a CardDescription. Use the `Target`, `BookOpen`, and `Bot` icons."

You can feed these prompts into v0.dev to get a great starting point for the UI.

I have now fully incorporated your requests into our plan. I am ready to start the build process myself.

When you are ready, please say __"toggle to__

scalytics

Of course. This is an excellent, comprehensive vision for a powerful B2B SaaS platform. Let's architect the entire concept for Enably: The Go-to-Market Architect.

Here is a detailed breakdown covering the platform's structure, features, technical architecture, and user experience, broken down into the separate pages and functions you described.

1. High-Level Vision & Core Concept

Enably is not just a tool; it's an AI-powered GTM co-pilot. It transforms the chaotic, manual process of building a sales engine into a guided, intelligent, and automated experience. It empowers founders, sales leaders, and reps to build, execute, and optimize their go-to-market strategy from a single, centralized platform.

Core Value Proposition: Move from GTM guesswork to a data-driven, AI-accelerated sales machine.

2. Platform Architecture (High-Level)

This illustrates how the different components interact.

+-----+	
	FRONTEND (Next.js)
	- Dashboard & Central Agent UI
	- GTM Architect Workflow UI
	- ICP, Playbook, Messaging Pages
	- SDR Co-Pilot Interface
	- Custom Agent Builder UI
+-----+	
(API Calls)	
+-----+	
	BACKEND (Supabase)
	- API Gateway (Edge Functions)
	- User Authentication (Supabase Auth)
	- Database (Postgres for user data, ICPs, etc)
	- Vector Store (pg_vector for knowledge bases)
	- File Storage (Supabase Storage for uploads)
+-----+	
(Secure API Calls)	
+-----+	
	AI LAYER
	- OpenAI Assistants API (Core conversation)
	- Vercel AI SDK (Frontend streaming & UI)
	- LangChain/LlamaIndex (Data processing/RAG)

	- Embedding Models (Text-to-vector)	
+	-----+	+
	(API Integrations)	
+	-----+	+
	THIRD-PARTY SERVICES	
	- CRM (Salesforce, HubSpot)	
	- Data Enrichment (Clearbit, Apollo.io)	
	- Lead Gen (LinkedIn Sales Nav, etc.)	
+	-----+	+

3. Detailed Page & Feature Breakdown

Here are the distinct sections/pages of the Enably application.

Page 1: The Dashboard / Central Hub

This is the user's mission control.

- UI: Clean, modern, and widget-based. Users can customize their view.
- Central Agent: A prominent, persistent chat interface, always available. This is the "main" assistant.
 - Function: Answers general questions ("How do I create a new ICP?"), provides platform support, gives high-level summaries ("Show me our top 3 performing email templates this week").
 - Tech: Powered by the OpenAI Assistants API with a general knowledge base about the Enably platform itself.
- Key Widgets:
 - GTM Progress: A visual tracker showing completion status of ICP, Playbook, etc.
 - Active Prospects: A feed of leads the SDR Co-Pilot is working on.
 - Key Metrics (KPIs): High-level view of metrics defined in the Sales Playbook (e.g., Emails Sent, Meetings Booked).
 - Quick Actions: "Create New ICP," "Start Prospecting," "Generate Report."

Page 2: The GTM Architect (Onboarding & Setup)

This is the core guided workflow to build the foundational assets.

- UI: A step-by-step, conversational wizard. It feels like an interview with a GTM expert.
- Process:
 1. Introduction: The Architect introduces itself and the process.
 2. Company Profile: Asks for basic company info, product details, and value proposition. Allows document uploads (pitch decks, one-pagers).
 3. Target Market: A conversational flow to define the Ideal Customer Profile (ICP).

- *"Let's define your first ICP. What industry are you targeting?"*
 - *"What's the typical company size? (e.g., 50-200 employees)"*
 - *"Who is the key decision-maker we need to reach? (e.g., VP of Engineering)"*
4. Asset Generation: Based on the inputs, it directs the user to the dedicated modules (ICP Workbench, Playbook Builder) to flesh out the details.

Page 3: ICP Workbench

A dedicated space to create, refine, and manage Ideal Customer Profiles.

- UI: A document-style editor on one side, and an AI chat/tool panel on the other.
- Features:
 - ICP Document: A living document generated by the AI, broken into sections:
 - ICP Definition: Company size, industry, geography, tech stack.
 - User Personas: Detailed profiles of key contacts (e.g., "Engineering Manager Emily," "CFO Charles"). The AI generates these with goals, pain points, and motivations.
 - Use Cases & Pain Points: Maps product features to specific customer problems.
 - Industry Research: AI agent performs web searches to pull in market trends, key competitors, and industry-specific language.
 - Unique Selling Proposition (USP): AI helps draft and refine USPs tailored to *this specific ICP*.

Page 4: Sales Playbook Builder

Where strategy becomes actionable.

- UI: A modular, wiki-style interface. Think Notion, but purpose-built for sales.
- Modules (Generated & Editable):
 - Team Structure: Defines roles (BDR, AE) and responsibilities.
 - Sales Process: Visual flowchart of the sales cycle (e.g., Prospecting -> Discovery -> Demo -> Proposal -> Close).
 - BDR/AE Activities: Daily/weekly SOPs. *"BDRs will send 50 personalized emails and make 20 calls per day."*
 - Metrics & KPIs: Defines what to track (e.g., Open Rates, Reply Rates, Meetings Booked, Conversion Rates). The AI can suggest industry-standard benchmarks.
 - Tool Stack: Recommends and documents how to use tools (CRM, LinkedIn Sales Nav, Enably itself).
 - AI Best Practices: A dedicated section on how to leverage Enably and other AI tools for prospecting, personalization, and follow-up.

Page 5: The Messaging Mage

The central repository for all sales communication.

- UI: A filterable library of templates and snippets.
- Content Types:
 - Email Templates: For cold outreach, follow-ups, post-demo notes. Includes variables like {firstName} and {companyName}.
 - Phone Scripts: Opening lines, objection handling, voicemail scripts.
 - DM Snippets: Short, concise messages for LinkedIn and other social platforms.
- AI Integration:
 - Generative AI: "Generate 3 cold email templates for our 'VP of Engineering' persona, focusing on the pain point of 'slow development cycles'."
 - Performance Analysis: (Future) Integrates with CRM/email tools to show which templates have the best open/reply rates.

Page 6: Presentation Studio (Future Module)

Automates the creation of sales collateral.

- UI: Template-based presentation builder.
- Function:
 1. User selects an ICP and a Use Case.
 2. The AI pulls the relevant company info, persona pain points, USPs, and case studies.
 3. It automatically generates a draft sales presentation (e.g., in Google Slides or as a downloadable PPTX) with a logical flow: Title -> Problem -> Solution -> How it Works -> Social Proof -> Next Steps.

4. Core Functions & Assistants

These are not pages, but cross-platform functionalities.

A. The SDR Co-Pilot (The Active Agent)

This is the proactive assistant for daily sales activities.

- UI: A dedicated panel or view, often used alongside the CRM or LinkedIn.
- Core Loop:
 - Lead Generation & Prospecting: "Find 20 companies in the fintech industry in New York with 100-500 employees." The agent uses integrated tools to build a prospect list.

- **Enrichment:** Once a list is built, the agent enriches it with contact info, job titles, and LinkedIn profiles.
- **Actionable Tasks:** It presents the SDR with a task list: "Send personalized email to Sarah at Acme Corp."
- **AI-Assisted Execution:** When the SDR clicks the task, the Co-Pilot opens a composer with the best email template from the Messaging Mage, and uses AI to personalize the first line based on the prospect's LinkedIn profile.
- **CRM Integration:**
 - **Update:** "Log this email and set a follow-up task in 3 days." The Co-pilot executes this via CRM API.
 - **Read:** "What's the status of our deal with Globex Corporation?" The co-pilot queries the CRM and provides the answer in chat.

B. Agent Factory (Advanced Feature)

Allows users to create their own specialized assistants.

- **UI:** A simple, no-code interface.
- **Process:**
 1. **Name Your Agent:** e.g., "Competitor Research Agent."
 2. **Define its Goal:** "This agent's job is to find and summarize information about our key competitors."
 3. **Upload Knowledge Base:** User uploads documents (competitor battle cards, market reports). Supabase `pg_vector` stores these as embeddings.
 4. **Grant Tools:** User can grant the agent access to specific tools, like "Web Search."
- **Backend:** This creates a new `Assistant` via the OpenAI API, links it to the uploaded knowledge files (Retrieval-Augmented Generation), and defines its available `tools` (functions).

5. Technical Stack & Implementation Details

- **Frontend:**
 - **Framework:** Next.js 14 (App Router) for its powerful routing, Server Components, and API route capabilities.
 - **UI Components:** Shadcn/ui built on top of Tailwind CSS. This provides beautiful, accessible, and highly composable components out-of-the-box, perfect for a modern SaaS look.
 - **State Management:** Zustand or React Query for managing server state and caching.
 - **AI Chat UI:** Vercel AI SDK is essential. It provides hooks (`useChat`, `useAssistant`) that handle streaming responses, managing conversation history, and calling tool functions seamlessly.
- **Backend:**

- Platform: Supabase. It's the perfect choice as it provides:
 1. Database: Postgres DB for all structured data (users, organizations, playbooks, etc.).
 2. Vector Store: `pg_vector` extension for storing embeddings for RAG (knowledge bases).
 3. Authentication: Built-in, secure user management.
 4. Storage: For user-uploaded documents (pitch decks, CSVs).
 5. Edge Functions: Serverless functions for handling API calls to OpenAI and other third parties, keeping secrets secure.
- AI Layer:
 - Core Engine: OpenAI Assistants API. This is critical for its three main features:
 1. Persistent Threads: Manages the entire conversational history automatically.
 2. Knowledge Retrieval: Powers the RAG for the ICP and custom agents.
 3. Function Calling: The magic that allows the assistant to interact with the real world (e.g., `updateCRM`, `searchWeb`, `getProspects`).
 - Data Processing: LangChain.js or LlamaIndex.ts can be used within Supabase Edge Functions for more complex data chunking, processing, and agentic workflows before interacting with the OpenAI API.
- Integrations (via Function Calling):
 - Your backend functions will act as the `tools` for the OpenAI Assistant.
 - `updateHubSpot(contactId, data)`
 - `getSalesforceLead(email)`
 - `searchLinkedIn(query)`
 - These functions will use the respective platform's Node.js SDKs or REST APIs, authenticated with user-provided API keys stored securely in your database.

Tab 4

Purpose

Purpose

Prompts and copy blocks to generate a world-class **homepage** and **in-app dashboard** for the GTM OS. Designed for v0.dev or any AI UI/code generator. Copy is modular; keep voice crisp, outcome-oriented, and founder/SDR friendly.

Global Creative Brief (use in every generation)

Product: A GTM operating system that helps teams define ICPs, personas, USPs, use-cases, sales bible, messaging templates, research/enrichment, and CRM sync—with an assistant that remembers context.

Brand Voice: Confident, helpful, concise. Challenger energy without jargon. Avoid buzzwords; use verbs and outcomes. Friendly to founders and SDRs.

Design Aesthetic: Clean, modern SaaS. Ample white space. Subtle glass/blur accents. Rounded-2xl, soft shadows, micro-interactions. Primary color = electric indigo (**#635BFF**) with slate neutrals. Accent = emerald (**#10B981**) for success.

Primary CTA: “Start the GTM Wizard”

Secondary CTA: “Explore the Demo Workspace”

Proof: Social tiles (logos) + short quantified outcomes.

Navigation: Dashboard · Wizard · Playbook · Messaging · Research · Hub · Integrations · Assistant · Settings

Accessibility: Min contrast ratio 4.5:1; focus rings visible; keyboard navigable; prefers-reduced-motion friendly.

Prompt — Homepage (System + User)

SYSTEM

You are a senior product designer and frontend engineer. Generate a production-ready Next.js + Tailwind homepage with shadcn/ui components. Output a single React page (App Router) that is accessible, responsive, and fast. Include semantic HTML, ARIA where needed, and mobile-first layout. Use a clear content hierarchy and persuasive copy tailored to a GTM operating system.

USER

Build the **Homepage** for “GTM OS”. Requirements:

1. **Hero** (above the fold)
 - Headline: *Ship your GTM in days, not months.*
 - Subhead: *Define ICPs, write playbooks, generate outreach, and sync to CRM—backed by a memory-aware assistant.*
 - CTAs: Primary **Start the GTM Wizard**; Secondary **Explore the Demo Workspace**.
 - Visual: right-side mock dashboard card stack (ICP preview, Playbook progress, Assistant message). Use tasteful gradient background and subtle grid.
 - Trust: mini logo strip with tooltip captions.
2. **Outcomes Section** (“Why teams choose us”)
 - 3–4 cards with icon + stat + sentence:
 - *<15 min to first ICP*
 - *3+ exportable assets on Day 1*
 - *First cadence live in 30 min*
 - *CRM sync in your first session*
3. **Guided Wizard Section**
 - Stepper illustration with steps: ICP → Personas → USPs → Use-Cases → Messaging → Positioning → Review/Export.
 - Short explainer and CTA link to wizard.
4. **Sales Bible & Messaging**
 - Two side-by-side feature panels: Activity Targets/SOPs/Cadences and Template Generator with variable chips (e.g., `{{first_name}}`, `{{pain_point}}`).
5. **Research & Enrichment**
 - One wide panel showing company snapshot (summary, triggers) and a **Generate Email** button.
6. **Assistant with Memory**
 - Chat preview with context toggles (ICP / Playbook / Current Lead). Include message bubble example with citations.
7. **Integrations**
 - HubSpot primary; S3, Clearbit listed. Badges with short labels. Include note: *Salesforce coming soon.*
8. **Security & Compliance**
 - Bullet list: row-level access, encrypted secrets, signed uploads, rate limiting.
9. **Footer**
 - Product, Company, Resources columns; social; copyright.

Design Tokens (Tailwind)

- Primary: #635BFF
- Accent (success): #10B981
- Gray scale: slate (Tailwind)
- Radius: rounded-2xl
- Shadows: shadow-lg shadow-slate-900/5
- Container widths: max-w-7xl with px-6 md:px-8.

Micro-interactions

- Button hover translate-y-0.5 + shadow intensify.
- Cards subtle hover ring (ring-1 ring-slate-200).
- Reduce motion if prefers-reduced-motion.

Copy Blocks (inject as literals)

- Proof points:
 - “Time-to-first-ICP: **14:32** on average.”
 - “Users export **3.6** assets their first day.”
 - “From idea → cadence in **29 minutes**.”
- Security bullets: “Row-level auth by workspace”, “Encrypted integration secrets”, “Signed S3 uploads”, “Upstash rate limiting”.

Acceptance

- Lighthouse ≥ 95 performance on desktop, ≥ 90 a11y.
- CLS < 0.02.
- No layout shift on font load (use system fonts or display=swap).

Prompt — Dashboard (System + User)

SYSTEM

You are a senior product designer and frontend engineer. Generate a production-ready **Dashboard** page (in-app) using Next.js + Tailwind + shadcn/ui. The dashboard should summarize progress and provide fast access to core actions. Include skeleton states, empty states, and real-looking placeholders. It must be accessible and responsive.

USER

Build the **Dashboard** for logged-in users of GTM OS. Requirements:

1. **Page Header**
 - Title: *Dashboard*

- Subtitle: *Your GTM at a glance.*
- Primary CTA: **Start the GTM Wizard** (if no ICP found); else **Open Playbook**.
- 2. **Progress Overview (cards row)**
 - **GTM Foundation:** completion ring (ICP, Personas, USPs, Use-Cases, Messaging, Positioning).
 - **Playbook Readiness:** status chips (Activity Targets, SOPs, Cadences, Qualification).
 - **Integrations:** HubSpot, S3, Enrichment — show connected/not connected.
- 3. **Today's Focus**
 - List of 5 tasks pulled from activity targets (e.g., "Send 40 emails", "20 DMs", "30 calls"). Include quick actions (copy template, open generator, log to CRM). Empty state: "No tasks yet—set Activity Targets in your Playbook."
- 4. **Recent Documents**
 - Table with Title, Type (ICP/Persona/Playbook/Template), Updated, Actions (View, Export, Pin). Empty state card with CTA: "Generate your first ICP".
- 5. **Assistant Shortcut**
 - Compact chat input: placeholder *Ask the assistant to draft a cadence...*; recent suggestions chips.
- 6. **Research Quick Add**
 - Domain/URL input; on submit, create snapshot card and button **Draft Email from Triggers**.
- 7. **Right Rail (optional on xl+)**
 - Metrics mini-cards: *Assets exported this week, Meetings booked, Reply rate*. Use placeholders and clear units.

Variants (stateful)

- New workspace: show setup checklist widget (Connect CRM, Create ICP, Add first template).
- Returning user: show progress ring and last opened doc link.

Empty States (copy)

- "Let's kickstart your GTM. The wizard turns your inputs into ICPs, personas, and playbooks in minutes."
- "No recent docs yet. Create your **ICP** to unlock messaging variables."

Skeletons

- Use `animate-pulse` cards for loading; reserve layout space to prevent CLS.

Acceptance

- Keyboard reachability for all quick actions.
- Announce dynamic updates with `aria-live="polite"` where relevant.
- Responsive down to 320px; grid collapses to single column.

Copy Library (plug-and-play)

Homepage Headlines

- “Ship your GTM in days, not months.”
- “From zero to outbound in one afternoon.”

Subheads

- “Define ICPs, write playbooks, generate outreach, and sync to CRM—backed by a memory-aware assistant.”
- “Founders and SDRs move faster with a shared, living GTM.”

CTAs

- Primary: “Start the GTM Wizard”
- Secondary: “Explore the Demo Workspace”

Benefit Bullets

- “Time-to-first-ICP under 15 minutes.”
- “Auto-generate cadences, DMs, and call openers.”
- “Company snapshots with triggers in one click.”
- “Works with HubSpot; Salesforce next.”

Trust Snippets

- “Teams use GTM OS to compress strategy work from weeks to hours.”

Dashboard Microcopy

- “Good morning—ready to move 5 deals forward?”
- “Your playbook is 72% complete. Add Cadences to unlock the generator presets.”
- “Connect HubSpot to log activities automatically.”

Component Guidelines (for generators)

- Use **shadcn/ui**: Button, Card, Tabs, Badge, Dialog, Sheet, Progress, Table, Input, Textarea, Tooltip, Skeleton.

- Spacing: `space-y-6` per section; grid gaps `gap-6`.
 - Cards: `rounded-2xl border border-slate-200/80 shadow-sm hover:shadow-md transition`.
 - Headings: `text-3xl md:text-4xl font-semibold tracking-tight` for hero; `text-xl font-semibold` for section titles.
 - Icons: **lucide-react** (e.g., Target, Book, Bot, Database, PlugZap, Shield, BarChart3).
 - Animations: use **framer-motion** for fade/slide in on scroll; respect reduced motion.
-

Sample Placeholder Data (Dashboard)

```
{
  "foundation": {"complete": 0.72, "sections": {"ICP": true, "Personas": true, "USPs": true,
"UseCases": false, "Messaging": true, "Positioning": false}},
  "playbook": {"activityTargets": true, "sops": true, "cadences": false, "qualification": "MEDDICC"},
  "integrations": {"hubspot": false, "s3": true, "enrichment": false},
  "tasksToday": [
    {"type": "email", "count": 40},
    {"type": "dm", "count": 20},
    {"type": "call", "count": 30}
  ],
  "recentDocs": [
    {"title": "Mid-Market SaaS Security Buyers", "type": "ICP", "updated": "2025-08-09"},
    {"title": "CISO Persona v2", "type": "Persona", "updated": "2025-08-09"},
    {"title": "Outbound Cadence A", "type": "Playbook", "updated": "2025-08-08"}
  ]
}
```

Acceptance Checklist (UI & UX)

One-Shot “Make It” Prompt (for v0/dev or similar)

Paste this as a single prompt:

Build a responsive Next.js + Tailwind homepage and dashboard for a SaaS called **GTM OS** following the spec below. Use shadcn/ui and lucide-react. Aesthetic: clean, modern, rounded-2xl, subtle gradients, primary #635BFF, success #10B981. Accessibility first. Include hero with action CTAs, outcomes metrics cards, wizard stepper section, sales bible/messaging panels, research snapshot, assistant preview, integrations row, security bullets, and a structured footer. Then a signed-in dashboard with progress cards, today's tasks, recent docs table, assistant quick input, research quick add, and right-rail metrics. Provide production-ready React components for the App Router, with Tailwind classes, semantic HTML, aria labels, skeleton/empty states, and small framer-motion animations. Use the copy blocks provided verbatim where specified. Ensure Lighthouse ≥ 90 a11y. Output a single file per page, `page.tsx`, and include any small UI subcomponents inline.

Stretch: Dark Mode Palette (optional)

- Background: #0B1020
- Card: #0F152A
- Text: #E5E7EB
- Borders: #1E293B
- Accent: keep #635BFF, success #10B981 with 90% opacity.